LJMP and SJMP in 8051

LJMP and SJMP in 8051

- Both **LJMP (Long Jump)** and **SJMP (Short Jump)** are unconditional jump instructions used to transfer program control to another location.
- The main difference is the **jump range**.

SJMP

- **Short Jump**
- • Relative addressing - SJMP instruction contains an 8-bit signed offset. This offset is added to the Program Counter (PC) after the SJMP instruction has been fetched.
- • Range: -128 to +127 bytes
- • 2-byte instruction
- • Used for nearby jumps

LJMP

- Long Jump
- • Absolute 16-bit address
- • Range: 0000H–FFFFH
- • 3-byte instruction
- • Used for far jumps
- Example : LJMP 3000H

Comparison

- SJMP: Relative, 2 bytes, ±127 bytes
- LJMP: Absolute, 3 bytes, 64 KB range

Examples

- SJMP LOOP
- LJMP 3000H
- Use SJMP for loops; LJMP for distant routines.

Conditional jump

- **Conditional jump instructions** transfer program control **only if a specified condition is true**.
- If the condition is false, the program continues executing the next instruction.
- All conditional jumps in the 8051 use **relative addressing**, so the destination must be within **–128 to +127 bytes** from the next instruction.

Conditional Jump Instructions in 8051

8051 conditional jump instructions

Instructions	Actions
JZ	Jump if A = 0
JNZ	Jump if A ≠ 0
DJNZ	Decrement and Jump if A ≠ 0
CJNE A,byte	Jump if A ≠ byte
CJNE reg,#data	Jump if byte ≠ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

Example

- MOV A,#00H
- JZ NEXT
- MOV R0,#55H ; Skipped
- NEXT : MOV R1,#AAH

ACALL and LCALL in 8051

- Both **ACALL (Absolute Call)** and **LCALL (Long Call)** are **subroutine call instructions** in the 8051.
- They transfer program control to a subroutine and automatically save the **return address** on the stack.
- The subroutine returns to the calling program using the RET instruction.

Difference

- ACALL calls a subroutine within the same **2 KB page** using an 11-bit address and occupies **2 bytes**.
- LCALL can call a subroutine anywhere in the **64 KB** program memory using a 16-bit address and occupies **3 bytes**.

What is a Subroutine in 8051?

- A **subroutine** is a **separate block of program instructions** written to perform a specific task.
- Instead of writing the same code multiple times, you write it once as a subroutine and **call** it whenever needed.
- After the subroutine finishes its task, it returns control to the main program using the **RET (Return)** instruction.

Example

- MAIN:
- ...
- ACALL DELAY ; or LCALL DELAY
- ...
- END

- DELAY:
- ; Instructions to generate delay
- RET

Example

- Main Program
- ------------
- MOV A,#25H
- ACALL DELAY
- MOV R1,A
- |
- v
- Subroutine (DELAY)
- -----------------
- NOP
- NOP
- RET
- |
- +------------------->
- |
- v
- Continue with MOV R1,A